# Data and Algorithm Analysis

## Chapter 23 — All-Pairs Shortest Path

Thang Hoang

Department of Computer Science, Virginia Tech

# Outline

Introduction

All-Pairs Shortest Path

Floyd-Warshall Algorithm

# Table of Contents

Introduction

All-Pairs Shortest Path

Floyd-Warshall Algorithm

# Overview

ALL-PAIRS SHORTEST PATHS

**Input**: Given a directed graph $G = (V, E)$, weight function $w : E \to \mathbb{R}$, $|V| = n$. Assume that the vertices are indexed $1, 2, \ldots, n$.

**Output:** An $n \times n$ matrix $D = (d_{ij})$ where $d_{ij} = \delta(i, j)$ for all vertices $i$ and $j$.

We can invoke BELLMAN-FORD for each vertex,

- $O(V^2 E)$ (or $O(V^4)$ if $G$ is dense, i.e., $E = \Theta(V^2)$)

Or Dijkstra alg. per vertex if no negative-weight edges.

- $O(VE \lg V)$ with binary heap — $O(V^3 \lg V)$ if dense,
- $O(V^2 \lg V + VE)$ with Fibonacci heap — $O(V^3)$ if dense.

**Can we do it better**? Like in $O(V^3)$ time in all cases?

# Definitions — Edge Weight Matrix

## Definition (Edge Weight Matrix)

Let $G = (V, E)$ be a directed graph, where $V = \{1, 2, \ldots, n\}$. Given edge weights on the edges of $G$, we define an $n \times n$ **edge weight matrix** $W = (w_{ij})$ by

$$w_{ij} = \begin{cases} 0 & \text{if } i = j; \\ \text{edge weight of } (i,j) & \text{if } i \neq j \text{ and } (i,j) \in E; \\ \infty & \text{if } i \neq j \text{ and } (i,j) \notin E. \end{cases}$$

The $w_{ij}$ weights generalize the $w(i,j)$ function used with single-source shortest paths.

# Definitions — Shortest Paths

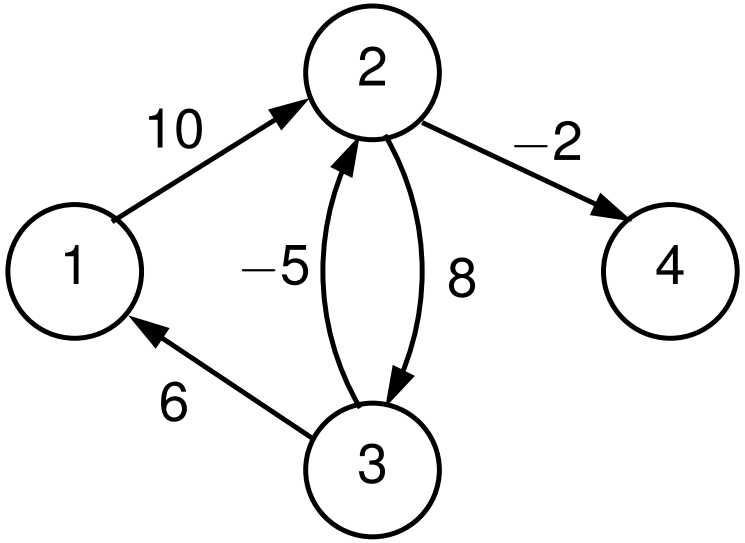
Let $u = v_0, v_1, \ldots, v_k = v$ be a path in $G$. The **weight** of $p$ is

$$w(p) = \sum_{i=1}^{k} w_{v_{i-1}, v_i}.$$

The **shortest path weight** from $u$ to $v$ is

$$\delta(u, v) = \begin{cases} \min\{w(p) \mid u \overset{p}{\rightsquigarrow} v\} & \text{if there is } p \text{ with } u \overset{p}{\rightsquigarrow} v; \\ \infty & \text{otherwise.} \end{cases}$$

# Example — Shortest Path




$$W = \begin{pmatrix} 0 & 10 & \infty & \infty \\ \infty & 0 & 8 & -2 \\ 6 & -5 & 0 & \infty \\ \infty & \infty & \infty & 0 \end{pmatrix}$$

# Table of Contents

Introduction

All-Pairs Shortest Path

Floyd-Warshall Algorithm

# A Problem

ALL-PAIRS SHORTEST PATH

**Input**: Directed graph $G = (V, E)$; edge weight matrix $W = (w_{ij})$.

**Output:** For each $u, v \in V$, $\delta(u, v)$.

# Recursive Solution

Let $l_{ij}^{(r)}$ = weight of shortest path $i \rightsquigarrow j$ that contains $\leq r$ edges.

- $r = 0$: shortest path exists iff $i = j$

$$l_{ij}^{(0)} = \begin{array}{ll} 0 & \text{if } i = j \\ \text{inf} & \text{if } i \neq j \end{array}$$

- $r \geq 1$:

$$l_{ij}^{(r)} = \min\{l_{ij}^{(r-1)}, \min\{l_j k^{r-1} + w_{kj} : 1 \leq k \leq n\}\}$$

($k$ ranges over all possible predecessors of $j$)

$$= \min\{l_{jk}^{(r-1)} + w_{kj} : 1 \leq j \leq n\}$$

(since $w_{jj} = 0 \forall j$)

Observation: when $r = 1$, $l_{ij}^{(1)} = w_{ij}$

# Recursive Solution

All simple shortest paths contain $\leq n-1$ edges.

$$\delta(i,j) = l_{ij}^{(n-1)} = l_{ij}^{(n)} = l_{ij}^{(n+1)} = \ldots$$

Compute a solution **bottom-up**

- Compute $L^{(1)}, \ldots L^{(2)}, \ldots, L^{(n-1)}$
- Start with $L^{(1)} = W$ (as $l_{ij}^{(1)} = w_{ij}$) and go from $L^{(r-1)}$ to $L(r)$:

EXTEND-SHORTEST-PATHS$(L^{(r-1)}, W, L^{(r)}, n)$

```
// Assume that the elements of L^(r) are initialized to ∞.
for i = 1 to n
    for j = 1 to n
        for k = 1 to n
            l_ij^(r) = min{l_ij^(r), l_ik^(r-1) + w_kj}
```

# Pseudocode — Recursive Solution

Compute each $L^{(r)}$:

```
SLOW-APSP(W, L^(0), n)
let L = (l_ij) and M = (m_ij) be new n × n matrices
L = L^(0)
for r = 1 to n − 1
    M = ∞        // initialize M
    // Compute the matrix "product" M = L · W.
    EXTEND-SHORTEST-PATHS(L, W, M, n)
    L = M
return L
```

Complexity?

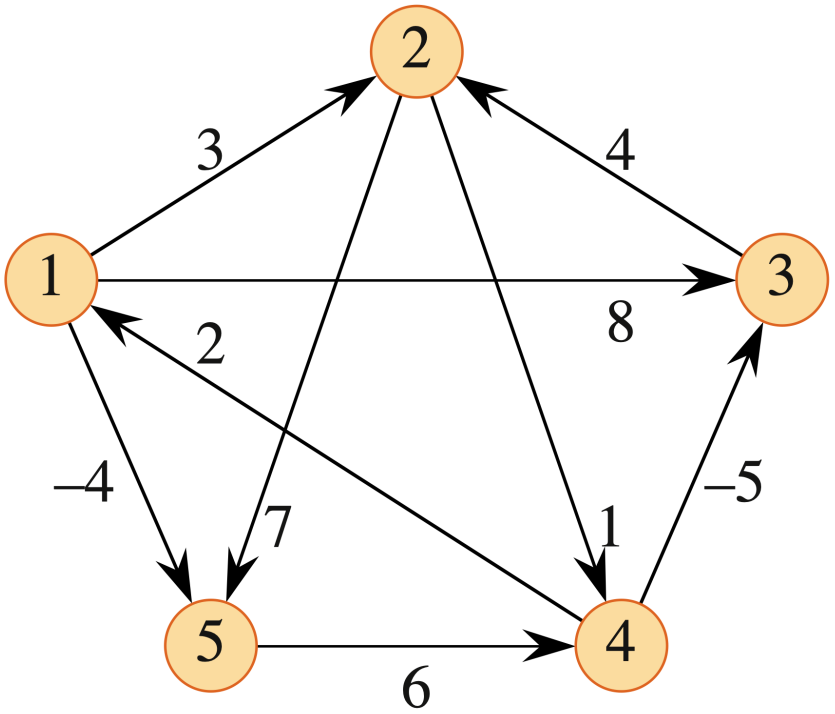
# All-Pairs Shortest Path and Matrix Multiplication

```
MATRIX-MULTIPLY(A, B, C, n)
for i = 1 to n                    // compute entries in each of n rows
    for j = 1 to n                // compute n entries in row i
        for k = 1 to n
            c_ij = c_ij + a_ik · b_kj   // add in one more term of equation (4.1)
```

EXTEND-SHORTEST-PATH is like matrix multiplication.

How?

# Example — All-Pairs Shortest Path




Do on board

# All-Pairs Shortest Path and Matrix Multiplication

Why do we care?

Our goal is to compute $L^{(n-1)}$ as fast as possible.

- No need to compute *all* the intermediate $L^{(1)}, L^{(2)}, L^{(3)}, \ldots, L^{(n-2)}$

Suppose we had a matrix $A$ and want compute $A^{n-1}$.

- Could do it fast by computing $A, A^2, A^4, \ldots$ (matrix squaring). If $A^r = A^{n-1}\ \forall r \geq n-1$, could just finish with $A^r$, where $r = 2^{\lg(n-1)}$.

# (Faster) All-Pairs Shortest Path Algorithm

```
FASTER-APSP(W, n)
let L and M be new n × n matrices
L = W
r = 1
while r < n − 1
    M = ∞          // initialize M
    EXTEND-SHORTEST-PATHS(L, L, M, n)    // compute M = L^2
    r = 2r
    L = M          // ready for the next iteration
return L
```

Complexity?

Can we do it even faster?

# Table of Contents

Introduction

All-Pairs Shortest Path

Floyd-Warshall Algorithm

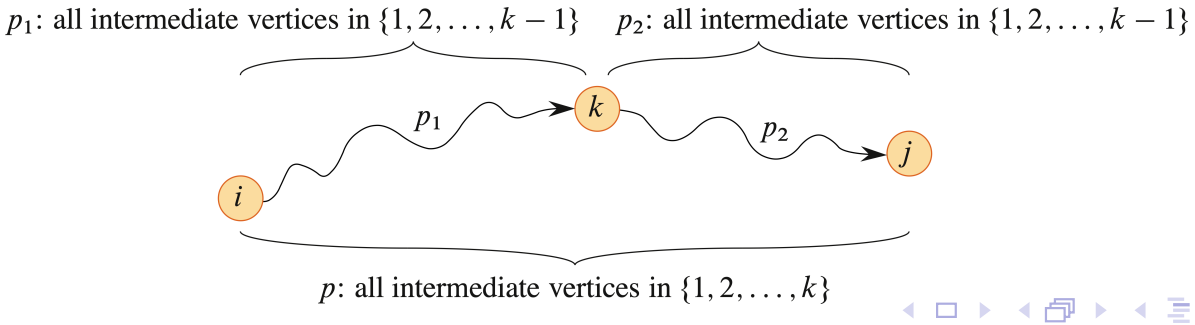
# Floyd-Warshall Algorithm

Another dynamic programming that can compute APSP faster by modeling the structure of APSP differently.

- For path $p = \langle v_1, v_2, \ldots, v_l \rangle$, an **intermediate** vertex is any vertex of $p$ other than $v_1$ or $v_l$.
- $d_{ij}^{(k)}$ = shortest-path weight for path $i \rightsquigarrow j$ with intermediate vertices in $\{1, 2, \ldots, k\}$.

Consider a short-test path $i \overset{p}{\rightsquigarrow} j$ with vertices in $\{1, 2, \ldots, k\}$:

- If $k$ is not an intermediate, all intermediate of $p$ are in $\{1, 2, \ldots, k-1\}$. Else, if $k$ is an intermediate vertex then



# Floyd-Warshall Algorithm

**Recursive Formulation**

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{if } k = 0 \\ \min\{d_{ij}^{(k)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} & \text{if } k \geq 1\} \end{cases}$$

$d_{ij}^{(k)}$ since cannot have intermediate vertices $\Rightarrow \leq 1$ edge.

Want $D^{(n)} = (d_{ij}^{(n)})$ since all vertices numbered $\leq n$.

# Floyd-Warshall Algorithm

```
FLOYD-WARSHALL(W, n)
D^(0) = W
for k = 1 to n
    let D^(k) = (d_ij^(k)) be a new n × n matrix
    for i = 1 to n
        for j = 1 to n
            d_ij^(k) = min{d_ij^(k-1), d_ik^(k-1) + d_kj^(k-1)}
return D^(n)
```

Complexity?

# Computing Predecessors — Warshall Algorithm

Compute predecessor matrix $\Pi$ while computing $D$ matrices.

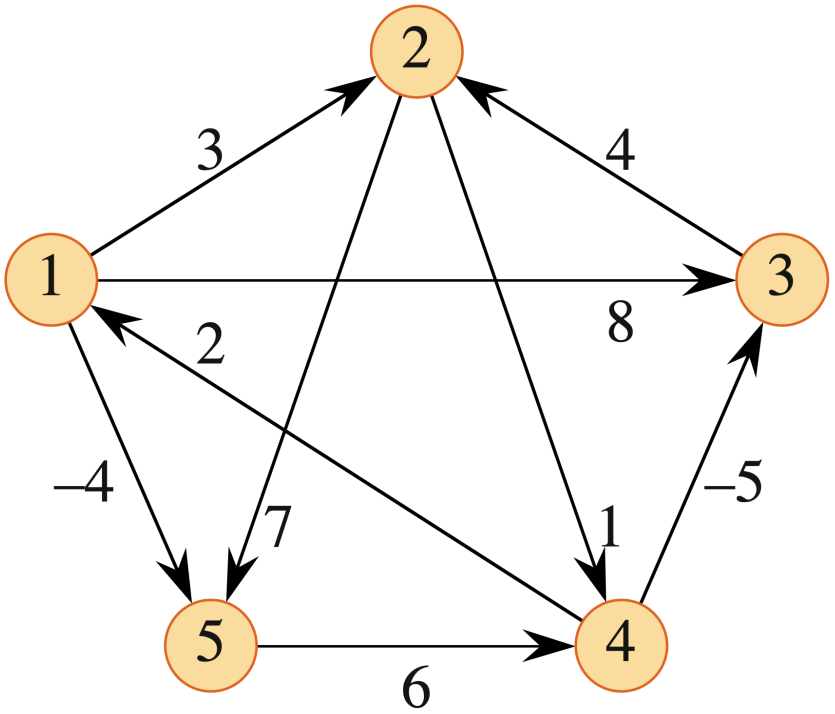
Let $\Pi^{(k)} = (\pi_{ij}^{(k)})$ for $k = 0, 1, \ldots, n$. Define $\pi_{ij}^{(k)}$ recursively

- For $k = 0$, shortest path of $i \rightsquigarrow j$ has no intermediate vertices

$$\pi_{ij}^{(0)} = \begin{cases} \text{NIL} & \text{if } i = j \text{ or } w_{ij} = \text{inf} \\ i & \text{if } i \neq j \text{ and } w_{ij} < \text{inf}\} \end{cases}$$

- For $k \geq 1$:
  - If $k$ is intermediate in shortest path of $i \rightsquigarrow j$, then $i \rightsquigarrow k \rightsquigarrow j$ ($k \neq j$). Select $j$'s predecessor as the predecessor of $j$ on shortest path of $k \rightsquigarrow j$ with all intermediate vertices $< k : \pi_{ij}^{(k)} = \pi_{kj}^{(k-1)}$.
  - Else, $k$ is not intermediate vertex. Keep the same predecessor as shortest path of $i \rightsquigarrow j$ with all intermediate vertices $< k : \pi_{ij}^{(k)} = \pi_{ij}^{(k-1)}$.

# Example — Floyd-Warshall Algorithm




Do on board

# Wrap-Up — All-Pairs Shortest Path

ALL-PAIRS SHORTEST PATH

**Input**: Directed graph $G = (V, E)$; edge weight matrix $W = (w_{ij})$.

**Output:** For each $u, v \in V$, $\delta(u, v)$.

**Matrix Multiplication Time Complexity:** $\Theta(n^3 \lg n)$

**Floyd-Warshall Time Complexity:** $\Theta(n^3)$